Normalization_Techniques

1. Deep learning
    1. Batch Norm
        1. Normalizing batch of inputs to eliminate internal covariate shift.
        2. torch.nn.BatchNorm1d(num_features) - Applies batch normalization over 2D or 3D
        3. For 4D inputs (N, C, H, W) - torch.nn.BatchNorm2d(num_features)
    2. Layer Norm
        1. Problems with Batch Norm
            1. Poor performance if the batch size is small, possible for high dimensional inputs
                1. But why not use bigger batch sizes? when having a very high resolution data for example width heigh resolution images, when flattened it becomes really huge in size. But because of memory constraints cannot fit in a lot of inputs together so we have to reduce the batch size. In such cases batch norm performs poorly.
            2. Running mean and average might not be the best thing to calculate for sequential algorithms like RNNs
        2. Layer normalization calculates mean and variance for each item within the batch
        3. Pytorch syntax - torch.nn.LayerNorm(normalized_shape)
    3. Weight Norm
        1. Normalizes weights of each layer torch.nn.utils.weight_norm(nn.Linear(20,40), name='weight')
    4. Group Norm
        1. Applies normalization over a mini-batch of inputs but splitted in groups of size num_channels/num_group
        2. torch.nn.GroupNorm(num_groups, num_channels)
    5. Instance Norm
        1. Calculates normalization parameters across individual channels/features for each input
        2. torch.nn.InstanceNorm1d(num_features)
        3. torch.nn.InstanceNorm2d(num_features)
2. 3D Vision
    1. Hartley Normalization
        1. To keep the eight-point solver numerically stable we normalize each point set:
            1. Translate points so their centroid is at the origin.
            2. Scale them so their average distance from the origin equals $\sqrt{2}$.
            3. The similarity transform applied to the first image is $T = \begin{bmatrix} s & 0 & -s\bar{u} \\ 0 & s & -s\bar{v} \\ 0 & 0 & 1 \end{bmatrix}$, $s = \frac{\sqrt{2}}{\text{mean distance from centroid}}$.
            4. Estimate the fundamental matrix F_{norm} with normalized points, then denormalize: $F = T_2^T F_{norm} T_1$. This reduces conditioning issues and prevents trivial solutions.
        2. 2D
            1. $T = \begin{bmatrix} s & 0 & -s\mu_{\tilde{x}} \\ 0 & s & -s\mu_{\tilde{y}} \\ 0 & 0 & 1 \end{bmatrix}$, where $s = \sqrt{\frac{2}{\sigma_{\tilde{x}}^2 + \sigma_{\tilde{y}}^2}}$, where $\mu_{\tilde{x}}$ and $\sigma_{\tilde{x}}^2$, and $\mu_{\tilde{y}}$ and $\sigma_{\tilde{y}}^2$ are the mean and variance of the $\tilde{x}$ and $\tilde{y}$ coordinates respectively (in homogeneous coordinates).
        3. 3D
            1. $U = \begin{bmatrix} s & 0 & 0 & -s\mu_{\tilde{X}} \\ 0 & s & 0 & -s\mu_{\tilde{Y}} \\ 0 & 0 & s & -s\mu_{\tilde{Z}} \\ 0 & 0 & 0 & 1 \end{bmatrix}$, where $s = \sqrt{\frac{3}{\sigma_{\tilde{x}}^2 + \sigma_{\tilde{y}}^2 + \sigma_{\tilde{z}}^2}}$
        4. Data normalize the points
            1. $\mathbf{x}_{\text{data\_normalized,i}} = T\mathbf{x}_i \forall i$
            2. $\mathbf{X}_{\text{data\_normalized},i} = U\mathbf{X}_i \forall i$

References:

1. 11-785 Deep Learning Recitation 4: Hyperparameter Tuning Methods, Normalization and Ensemble Methods.
2. Group norm paper.

3. Epipolar geometry, 3D vision, Elte.